Guía de Ejercicios N°8

EIE 434 – Programación 2 | Escuela de Ingeniería Eléctrica

Unidad: 1 **Semestre:** 1er Semestre 2026 **Docente:** Guillermo Cid Ampuero

Ayudante: Amaro Díaz

1. Objetivos

Esta guía tiene como objetivo funcionar como un **Ejercicio Integrador de Resumen**. Los estudiantes deberán aplicar y consolidar en un solo proyecto práctico todos los conocimientos adquiridos durante el curso: lógica algorítmica básica (ciclos `while` y condicionales), Programación Orientada a Objetos (diseño de clases y encapsulamiento estricto), manipulación de arreglos con **NumPy** y visualización de datos con **Matplotlib**.

2. Ejercicios

Ejercicio 1 – Proyecto Integrador: Motor de Simulación de Combate RPG

Usted ha sido asignado como ingeniero de software para desarrollar el núcleo lógico de un juego de rol por turnos. Para garantizar que la simulación sea robusta, segura y medible, construirá el sistema por fases, aplicando buenas prácticas de ingeniería de software y finalizando con un análisis gráfico de los datos del combate.

Fase 1: El Molde Base y Seguridad de Datos (POO y Encapsulamiento)

a) Defina una clase `Personaje`. Su constructor debe recibir e inicializar los atributos `nombre`, `hp_maximo` y `ataque_base`.

b) Implemente **Encapsulamiento Estricto**: Cree un atributo privado llamado `__hp_actual` (con doble guion bajo) que inicie valiendo lo mismo que el HP máximo. Además, inicialice una lista pública llamada `historial_hp` que contenga este valor inicial en su primera posición.

c) Cree un método `esta_vivo(self)` que retorne `True` si la vida actual es mayor a cero, y `False` en caso contrario.

d) Implemente el método `recibir_dano(self, cantidad)`. Este método es la **única** interfaz válida para restar vida al personaje. Agregue un condicional de validación: si la vida resultante es menor a cero, debe fijarse exactamente en 0 (la vida negativa no existe en este sistema). Finalmente, guarde el nuevo valor de vida en la lista `historial_hp`.

Fase 2: Lógica Discreta de Daño (Condicionales y Matemáticas)

e) Añada el método `atacar(self, objetivo, turno)`. En lugar de usar números aleatorios, el sistema será determinista. Utilice el operador módulo (`%`) sobre la variable `turno` para evaluar lo siguiente: si el turno es múltiplo de 3, el ataque es un 'Golpe

Crítico' y el daño final será el **doble** del ataque base. Si no lo es, el daño será el ataque base normal.

f) Imprima en consola un mensaje narrativo indicando quién ataca a quién y con cuánto daño (señalando si fue crítico). Luego, ejecute la acción llamando al método `recibir_dano` del objeto objetivo.

Fase 3: El Motor de Simulación (Ciclos)

g) En el bloque principal de su script (`if __name__ == '__main__':`), instancie dos personajes: un Héroe (ej. 100 HP, 15 de ataque) y un Dragón (ej. 150 HP, 12 de ataque). Inicialice una variable contadora `turno_actual` en 1.

h) Cree un ciclo `while` que represente el combate. Este ciclo debe ejecutarse únicamente mientras **ambos** personajes sigan vivos (use el operador `and`).

i) Dentro del ciclo, el Héroe ataca primero al Dragón pasando el `turno_actual`. Inmediatamente después, si el Dragón **aún sigue vivo**, este contraataca al Héroe. Al finalizar ambos movimientos, sume 1 al contador de turnos. Al salir del ciclo, imprima quién fue el ganador.

Fase 4: Análisis y Visualización (NumPy y Matplotlib)

j) Finalizada la simulación, importe `numpy` como `np` y convierta las listas de `historial_hp` de ambos personajes en arreglos de NumPy.

k) Genere un arreglo para el Eje X (los turnos jugados) utilizando `np.arange()` basándose en la longitud del historial de cualquiera de los personajes.

l) Importe `matplotlib.pyplot` como `plt`. Cree una figura y grafique ambas curvas de HP utilizando `plt.plot()`, asignando colores distintos (ej. azul para el héroe, rojo para el dragón) y etiquetas (`label`).

m) Agregue a su gráfico un título descriptivo, nombres a los ejes X e Y, una línea horizontal de referencia en el nivel 0 (`plt.axhline(0)`), la leyenda y la grilla. Muestre el gráfico final usando `plt.show()`.

Nota: Solución de la Clase: Repositorio del Curso